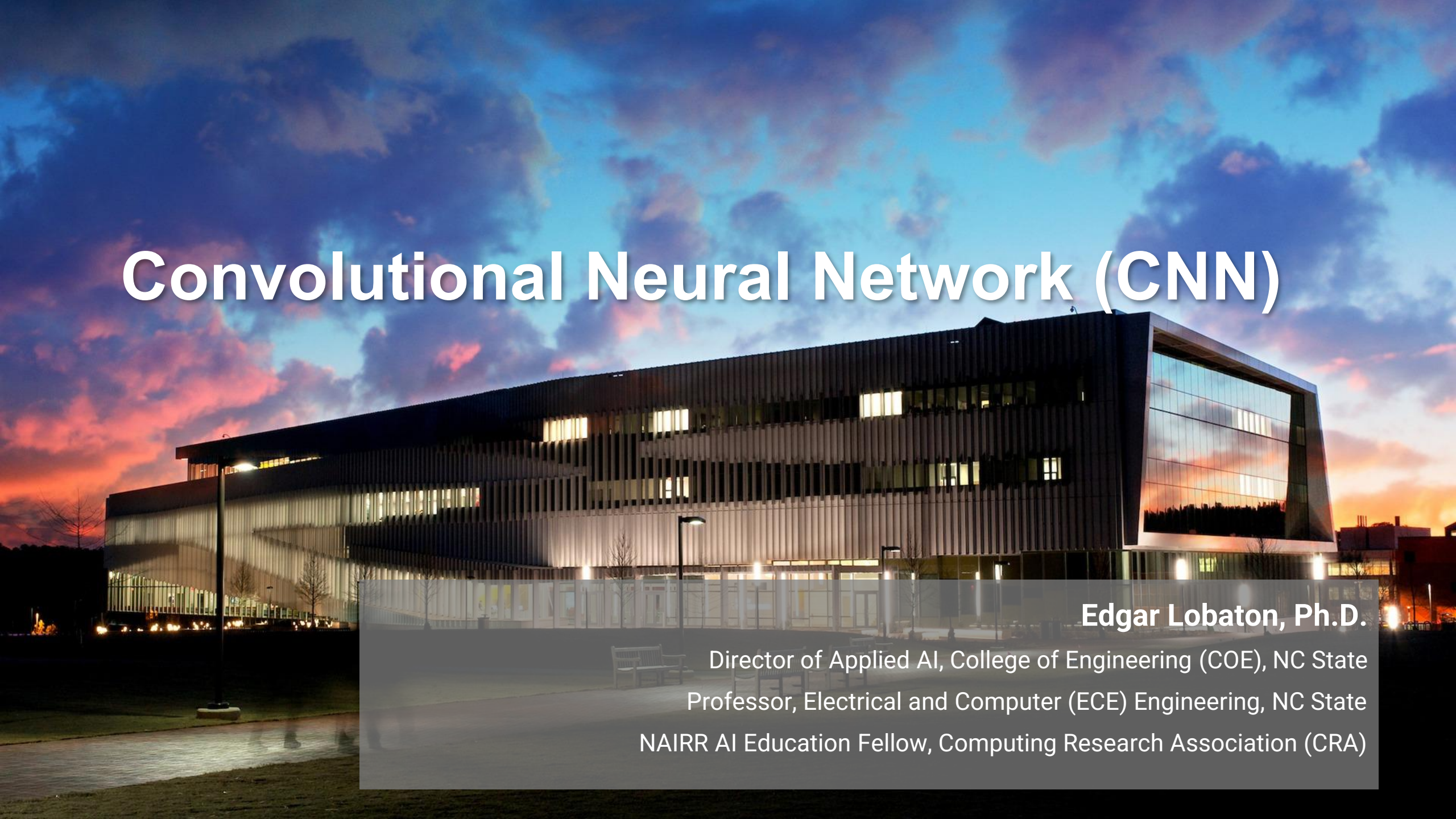


Convolutional Neural Network (CNN)

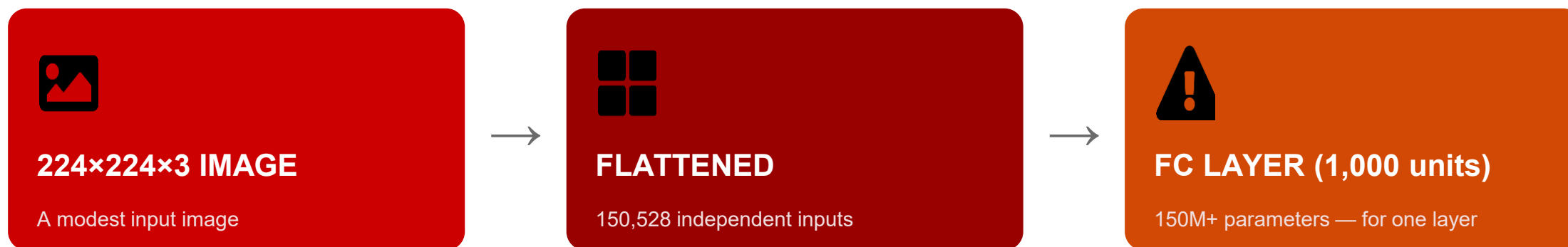
A photograph of a modern, multi-story building with a large glass facade, illuminated from within and reflecting the vibrant colors of a sunset sky. The sky is filled with clouds in shades of blue, orange, and pink. The building has a dark, angular roof and a facade of vertical slats. The foreground shows a paved area and some greenery.

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

Why Fully-Connected Layers Fail on Images

The problem isn't accuracy — it's scale and structure

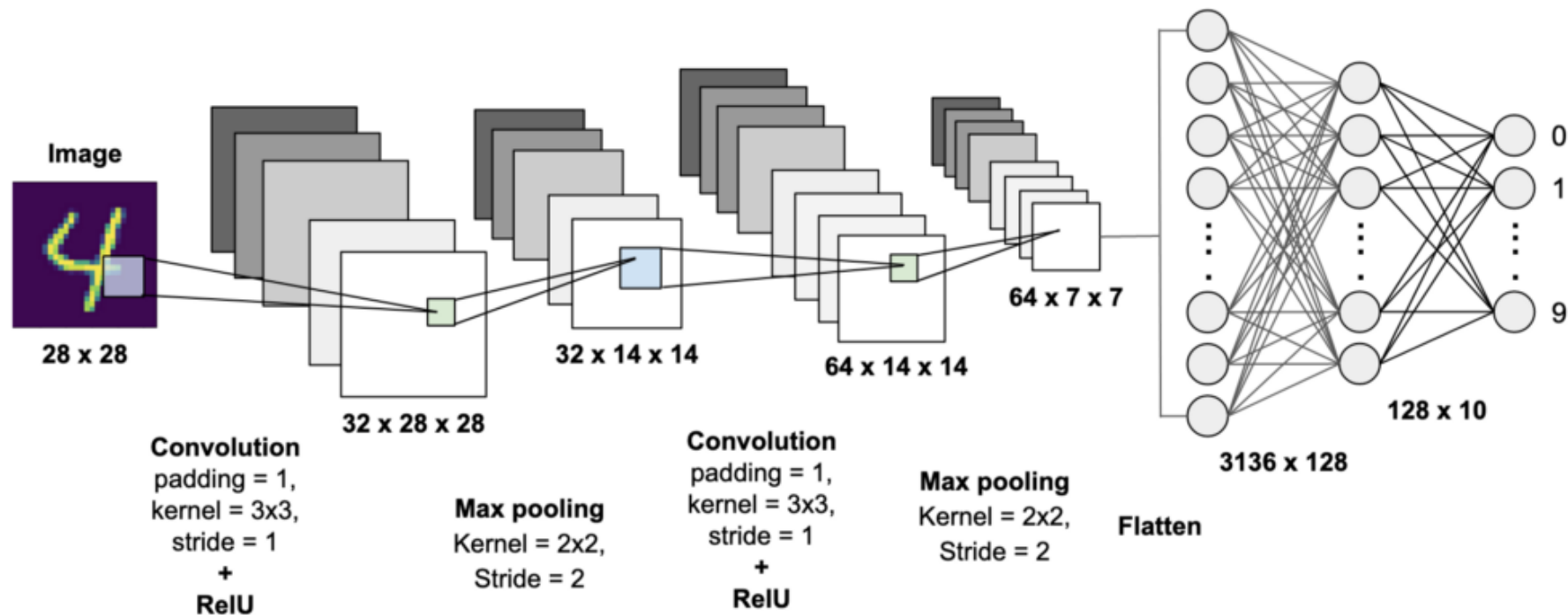


And parameter count is only half the problem: flattening also throws away spatial structure — nearby pixels are treated as completely independent, when they're highly related.

Convolutional layers fix both problems at once: far fewer parameters, and structure that respects spatial relationships.

The Architecture

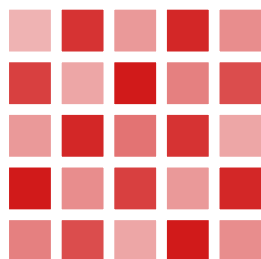
- Main Components: **Convolutional Layer** (convolution plus activation function), **Pooling** (subsampling), and Fully Connected Layers (i.e., an MLP)



The Core Operation: Convolution

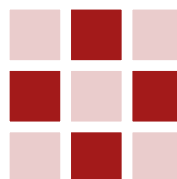
A small learned filter slides across the image, computing a local response

INPUT PATCH



*

KERNEL (3×3)



=

FEATURE MAP



Here, a vertical-edge kernel produces a strong response exactly where a vertical edge appears in the patch.



Local Receptive Fields

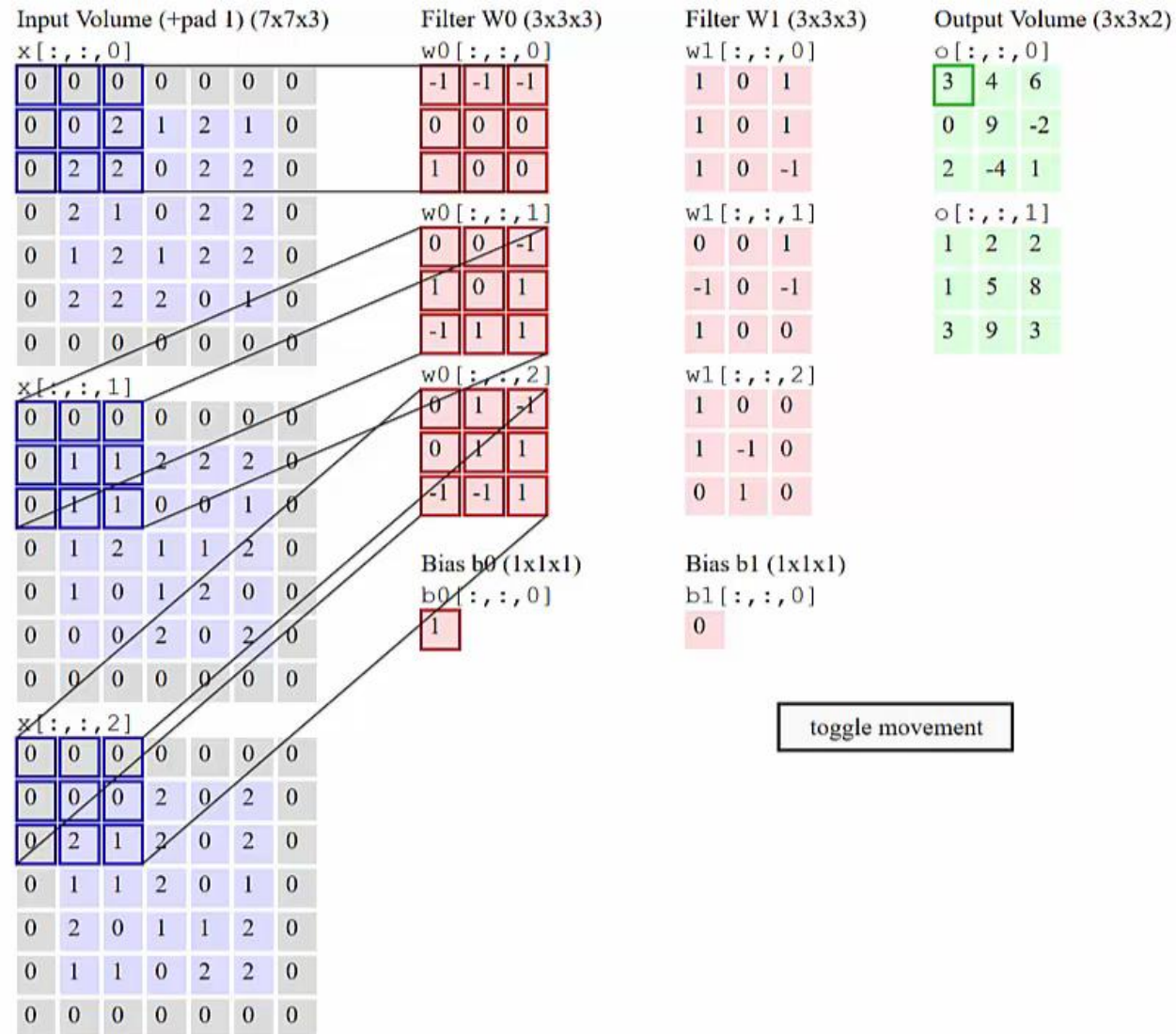
Each output value depends only on a small neighborhood — not the entire image.



Weight Sharing

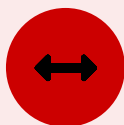
The same filter is reused at every position, instead of learning a separate weight per pixel.

The Core Operation: Convolution



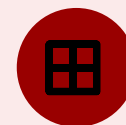
Stride, Padding & Pooling

Three supporting operations that shape how convolution behaves



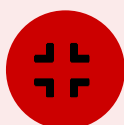
Stride

How far the filter moves at each step — a larger stride shrinks the output faster.



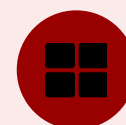
Padding

Controls what happens at the image border, so edge pixels aren't underrepresented.



Pooling

Downsamples feature maps (max or average), reducing computation.

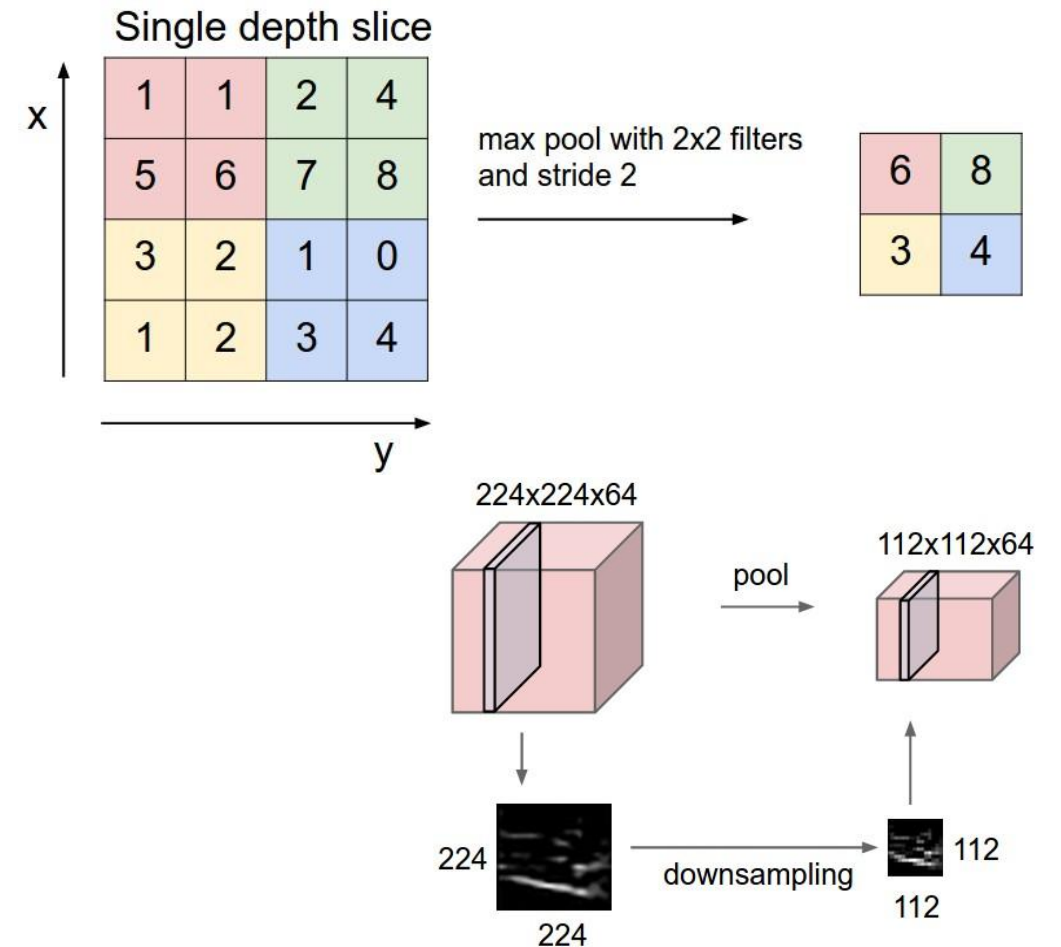


Why Pooling Helps

Adds translation invariance — a crack detector should still fire if the crack shifts a few pixels.

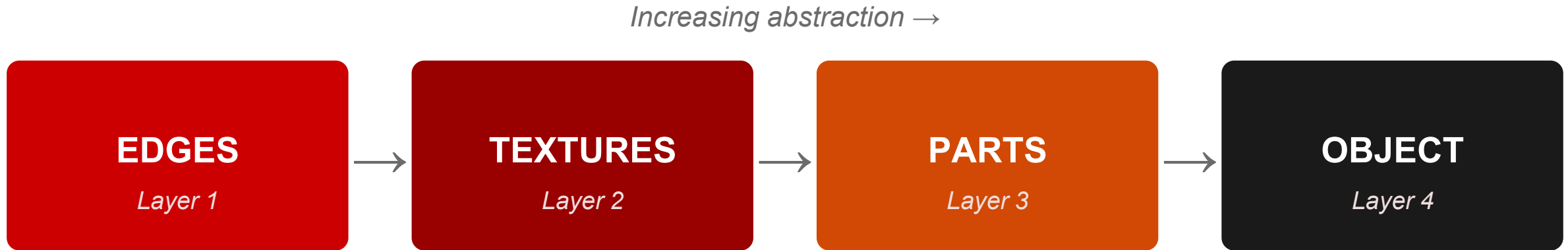
Pooling

- It provides a summary statistic of nearby outputs
- **Max pooling** is a common choice. It provides a level of invariance to local translation.
- We often just care if a feature (e.g., an eye when looking to detect a face) is present instead of focusing on the location.



Depth & Hierarchy

Stacking convolutional layers builds increasingly abstract representations

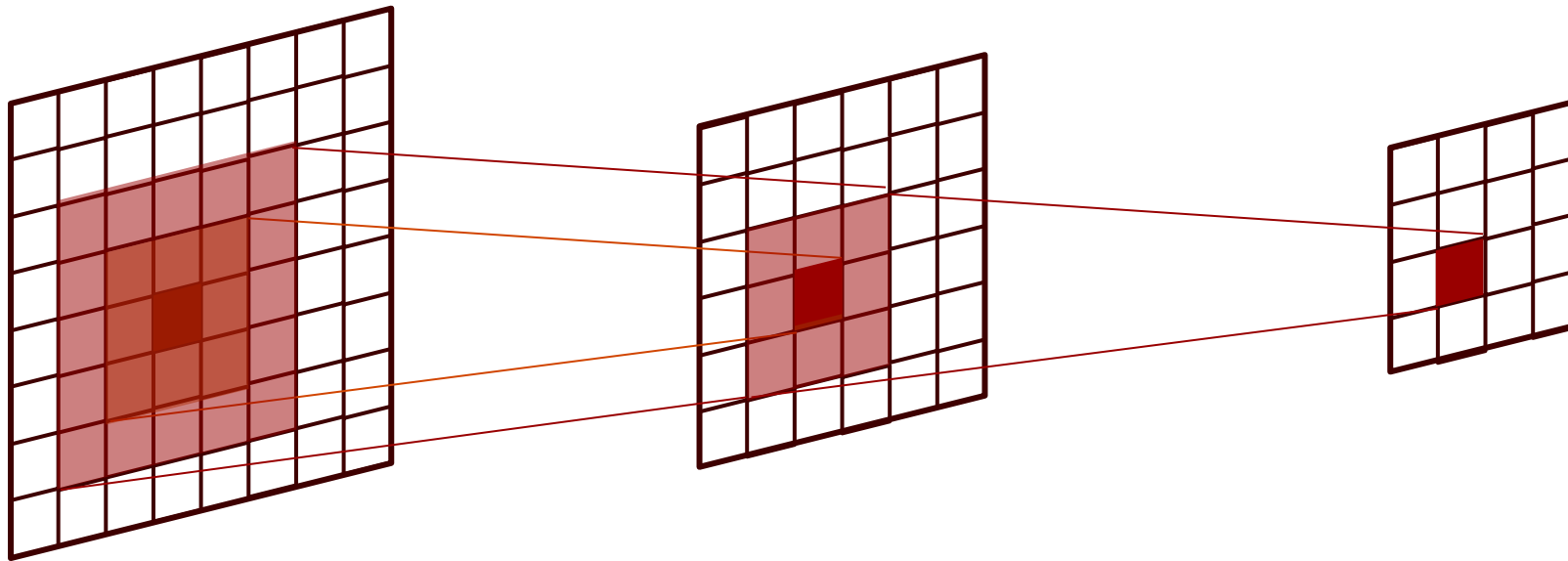


This is the same abstraction idea from the Deep Learning overview slide — now made concrete with an actual operation behind it.

Depth & Hierarchy

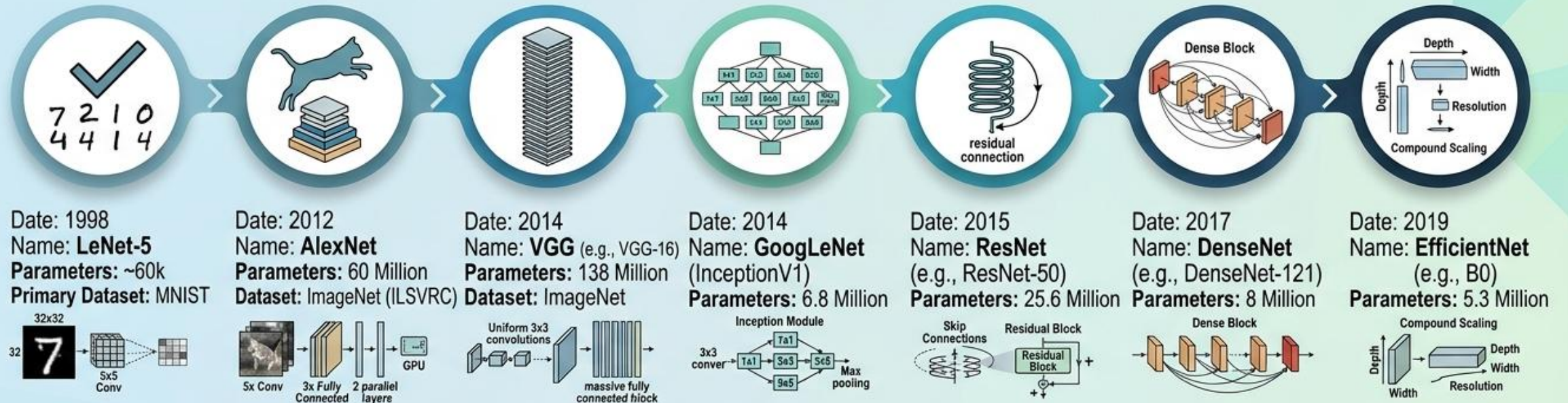
Stacking convolutional layers builds increasingly abstract representations

- Note that the deeper you go on a neural net, the more non-linearly you incorporate. However, for a CNN, we also have that the deeper you go, we get a larger support.
- Consider the convolutional layers with 3x3 kernels



Architecture Evolution: A Fast Pass

CONVOLUTION-DOMINANT ERA



EMERGING & CONVERGING ERA



CNN Engineering Applications

Where the automatic feature learning actually pays off



Pavement / Bridge Crack Detection

The CNN that finally does the job the hand-built HOG/SVM pipeline struggled with.



Structural Component Recognition

Identifying components in drone imagery, built up from the edges → parts hierarchy.



Thermal / RGB Fusion for Inspection

Combining modalities for rail and infrastructure inspection — your CRISI work.



General Visual Inspection

Any task where the defect is visual but its exact appearance varies too much to hand-code.

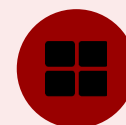
CNN Limitations

The honest counterweight — and the opening for what comes next



Needs Large Labeled Datasets

Strong performance typically requires substantial labeled training data.

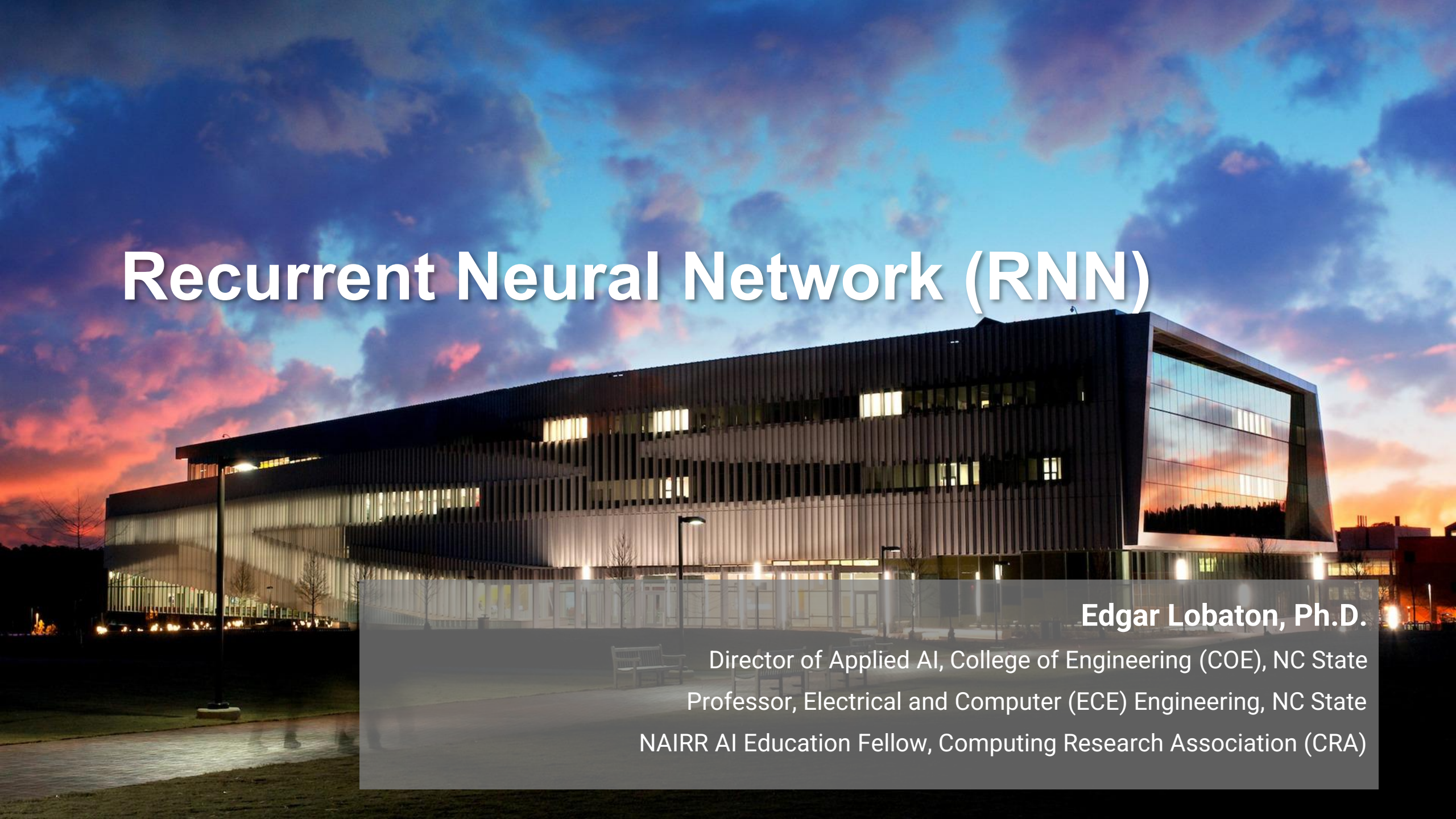


Assumes Fairly Fixed Input Structure

Built around a spatial grid — not naturally suited to variable-length data.

No native way to handle sequences: a single image, yes. A time-series of sensor readings, or a video, not natively — that's where recurrent networks come in.

Recurrent Neural Network (RNN)

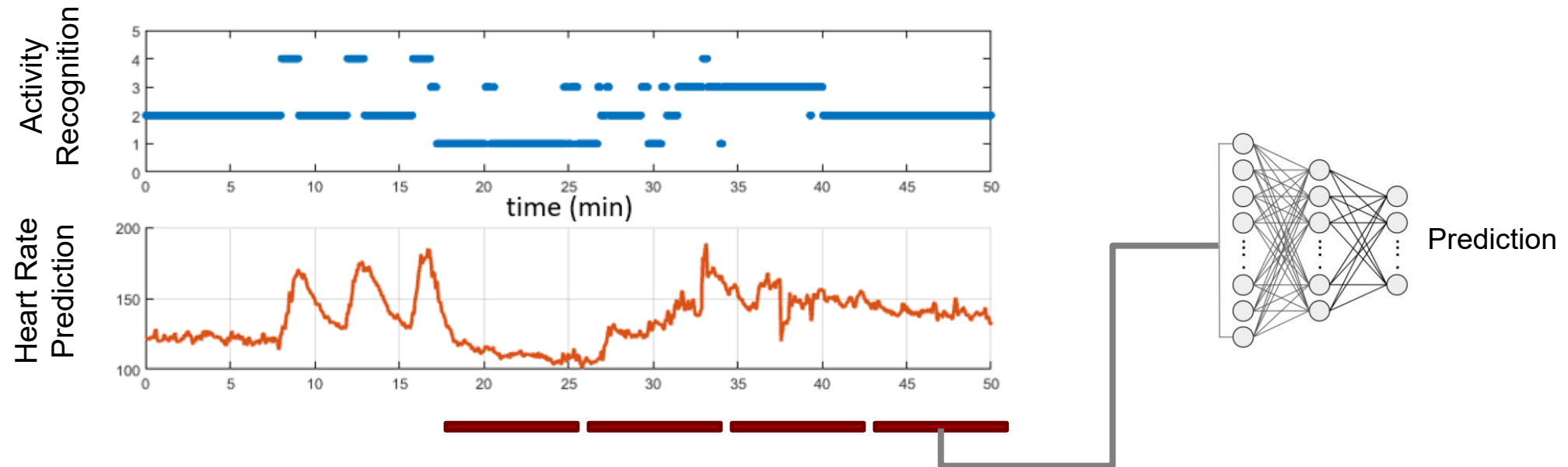
A photograph of a modern, multi-story building at dusk. The building features a large glass facade on the right side, which reflects the vibrant orange and pink colors of the sunset sky. The rest of the building has a dark, textured facade with horizontal slats. Several windows are illuminated from within, showing a warm yellow light. The sky is filled with soft, colorful clouds. In the foreground, there is a paved area and some greenery.

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

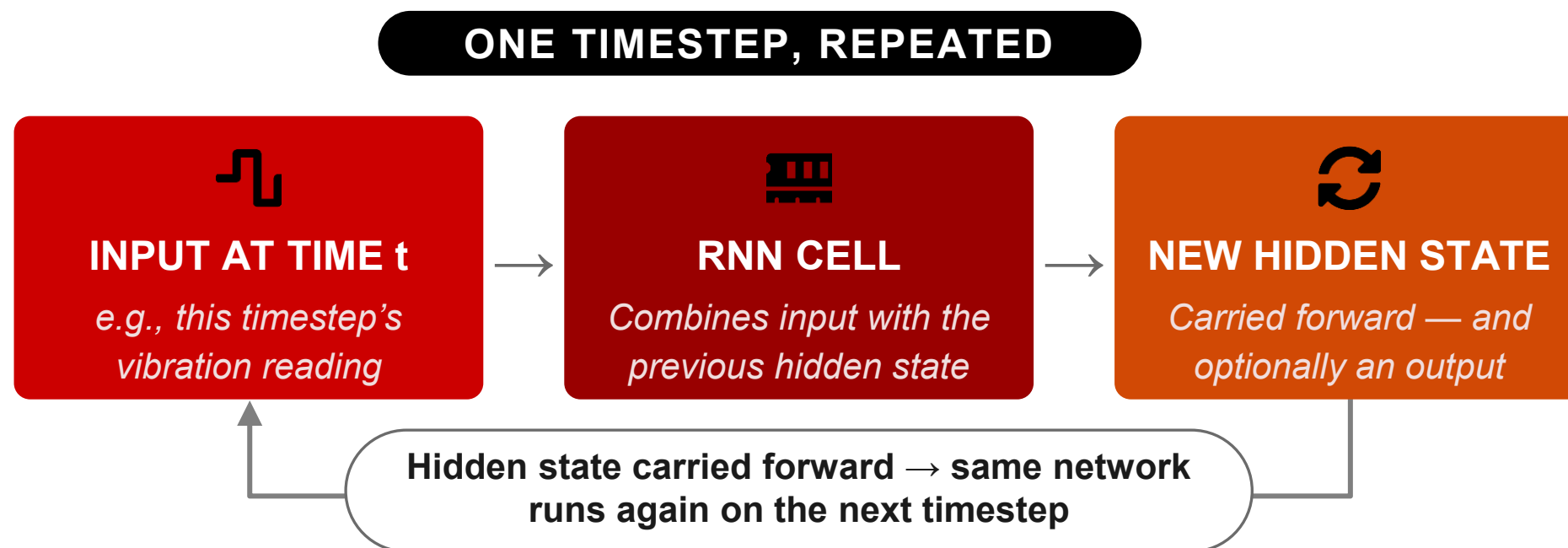
Predictive Models for Sequential Data

- We need something more specialized for time-series data (e.g., NLP, stock market, physiological responses, video sequences)
- MLPs and CNNs are an alternative. However, they do not exploit potential temporal information. They are memoryless.



The Core Idea: Hidden State Through Time

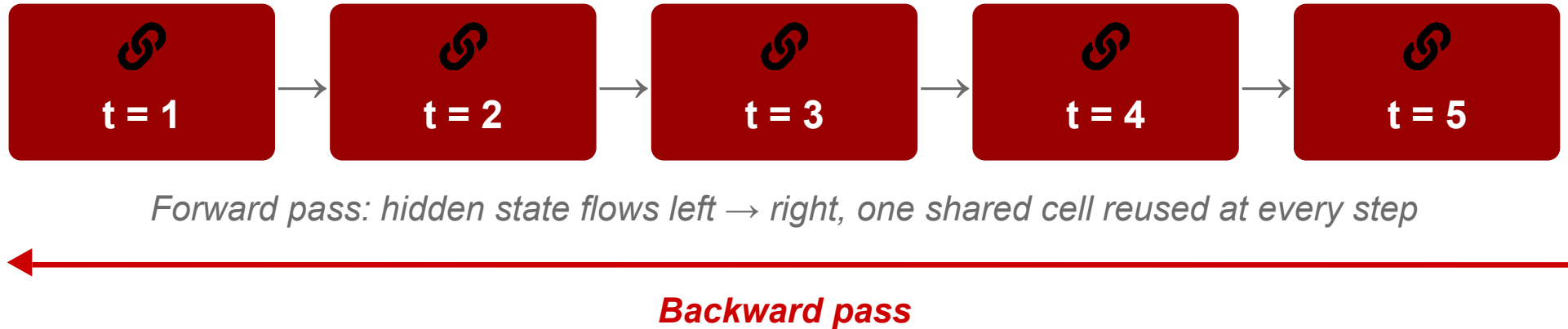
The same recurrent loop pattern from the LLM session — now carrying a learned state



Parameter sharing across time — the RNN version of the weight-sharing-across-space idea from convolution.

Unrolling & Backpropagation Through Time

The same cell, drawn out across every timestep



Conceptually only: the recurrent step is “unrolled” into this chain, and training backpropagates through the whole thing.

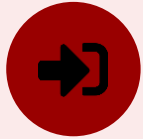
LSTM: Gating as the Fix

Conceptual view — learned gates control what the network remembers



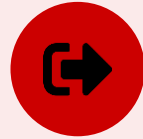
Forget Gate

Decides what to discard from the existing memory.



Input Gate

Decides what new information to add to memory.

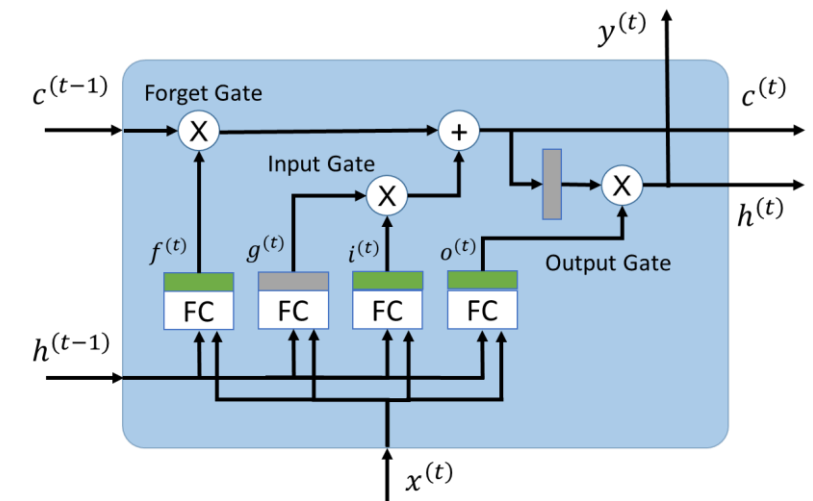


Output Gate

Decides what part of memory to expose as output.



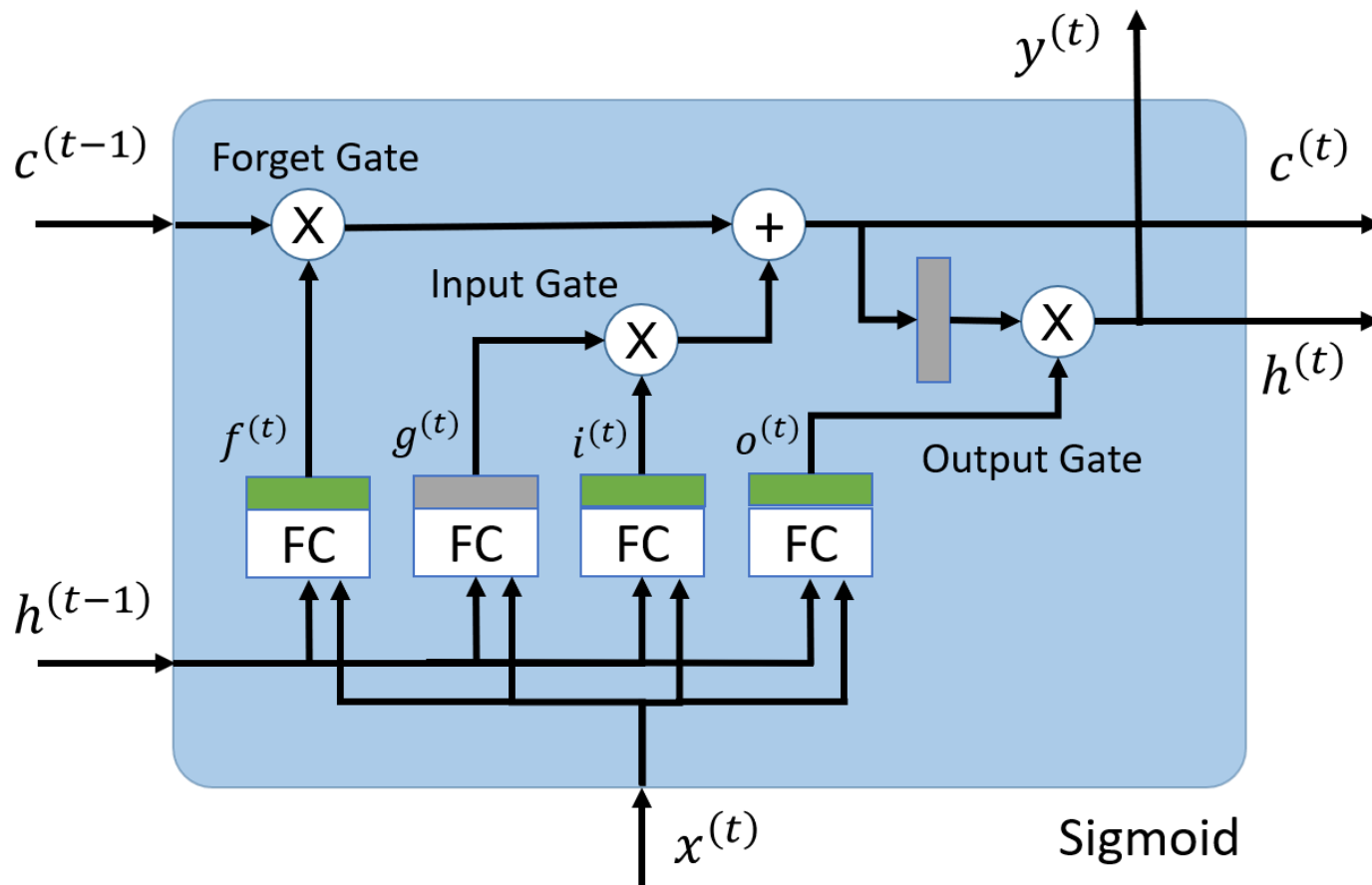
Cell State — a controllable long-term memory, instead of one that decays automatically



Not the full gate equations here — the takeaway is that gates give the network a choice about what to keep, rather than forgetting by default.

LSTM: Gating as the Fix

Conceptual view — learned gates control what the network remembers



$$g^{(t)} = \tanh(W_{xg} \cdot x^{(t)} + W_{hg} \cdot h^{(t-1)} + b_g)$$

$$i^{(t)} = \sigma(W_{xi} \cdot x^{(t)} + W_{hi} \cdot h^{(t-1)} + b_i)$$

$$f^{(t)} = \sigma(W_{xf} \cdot x^{(t)} + W_{hf} \cdot h^{(t-1)} + b_f)$$

$$o^{(t)} = \sigma(W_{xo} \cdot x^{(t)} + W_{ho} \cdot h^{(t-1)} + b_o)$$

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot g^{(t)}$$

$$y^{(t)} = h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

RNN Engineering Applications

Where sequence matters as much as the individual reading



Predictive Maintenance from Time-Series

Vibration or sensor streams where recent history predicts what comes next.



Fatigue / Failure Forecasting

Revisiting the fatigue-life scenario as a sequence problem, not a single-shot fit.



Structural Response Modeling

Modeling how a structure responds over time to a sequence of loads.



Anywhere Order Carries Information

If yesterday's reading changes how you interpret today's, this family applies.

Synthesis: Spatial vs. Sequential Structure

Two families, one underlying idea — build in the right assumption for the data

	CNN	RNN
Exploits	Spatial structure — nearby pixels relate	Sequential structure — nearby timesteps relate
Sharing mechanism	Same filter reused across space	Same cell reused across time
Core building block	Convolution + pooling	Hidden state passed forward
Known weakness	No native handling of sequences	Long-range dependencies decay (vanishing gradient)

Same underlying principle, different axis: share weights wherever the data has a repeating structure to exploit.

AutoEncoder (AE)

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

Why Learn Without Labels?

Everything so far has needed labels or a next-token target — what if you just have raw data?

THE SCENARIO

Years of vibration data from healthy pumps — but almost no labeled examples of the faults you actually care about.

THE AUTOENCODER IDEA

Skip labels entirely. Train the network to reconstruct its own input — the input itself is the training target.

This is self-supervised learning: the loss compares the reconstruction to the original input — no human-provided label required.

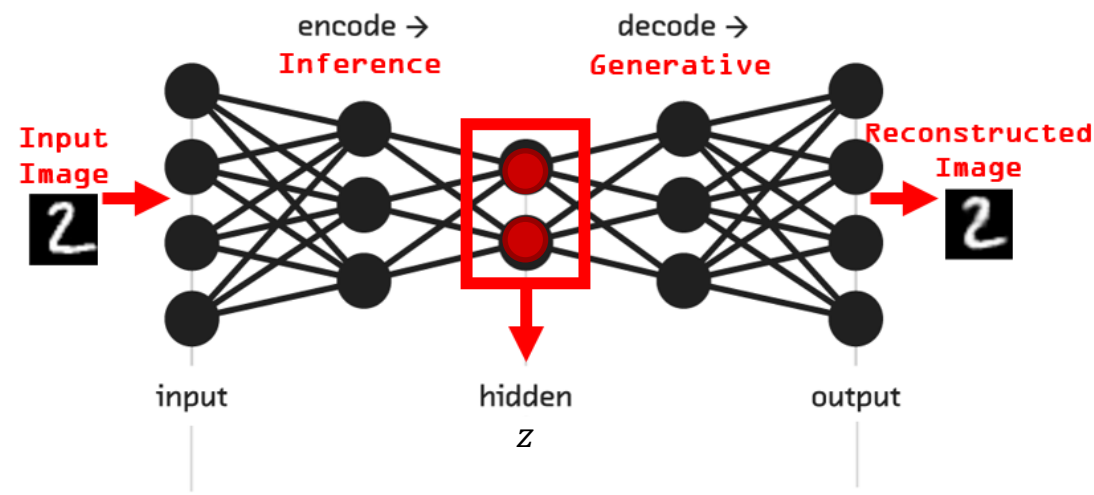
The Core Architecture: Encoder → Bottleneck → Decoder

Trained end-to-end to reconstruct its own input



The reconstruction loss (e.g., mean squared error) compares OUTPUT to INPUT directly — that comparison is the entire training signal.

***Notice the shape: wide → narrow → wide.
That narrow middle is doing all the work.***



Why the Bottleneck Matters

Without a constraint, the network could just learn to copy



NO REAL BOTTLENECK

If the latent code is as large as the input, the network can just learn the identity function — perfect reconstruction, zero useful structure learned.



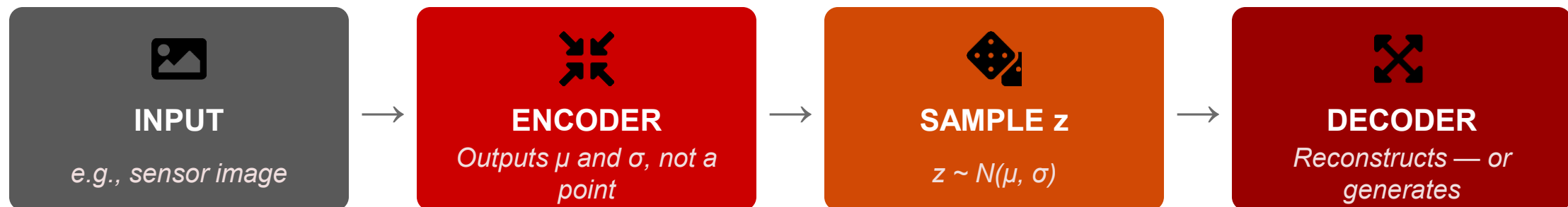
A REAL BOTTLENECK

A latent code far smaller than the input forces the network to discard noise and keep only the structure that matters most for reconstruction.

The bottleneck width is a knob: too wide and it copies, too narrow and it can't reconstruct at all.

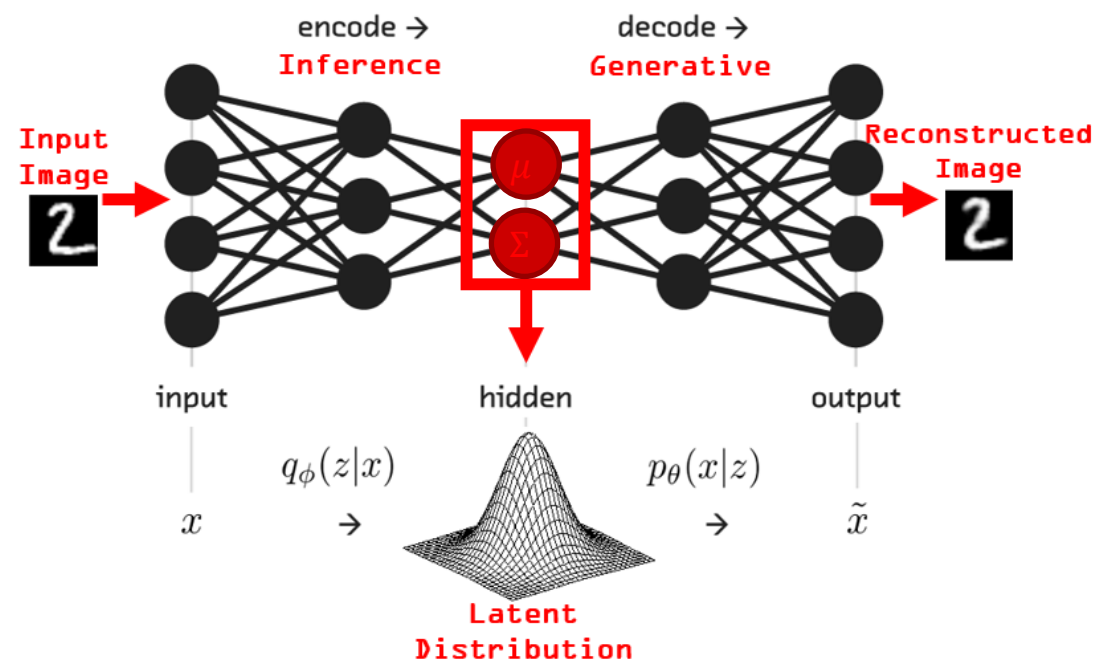
VAEs: A Probabilistic Latent Space

The variant that turns a compressor into a generator



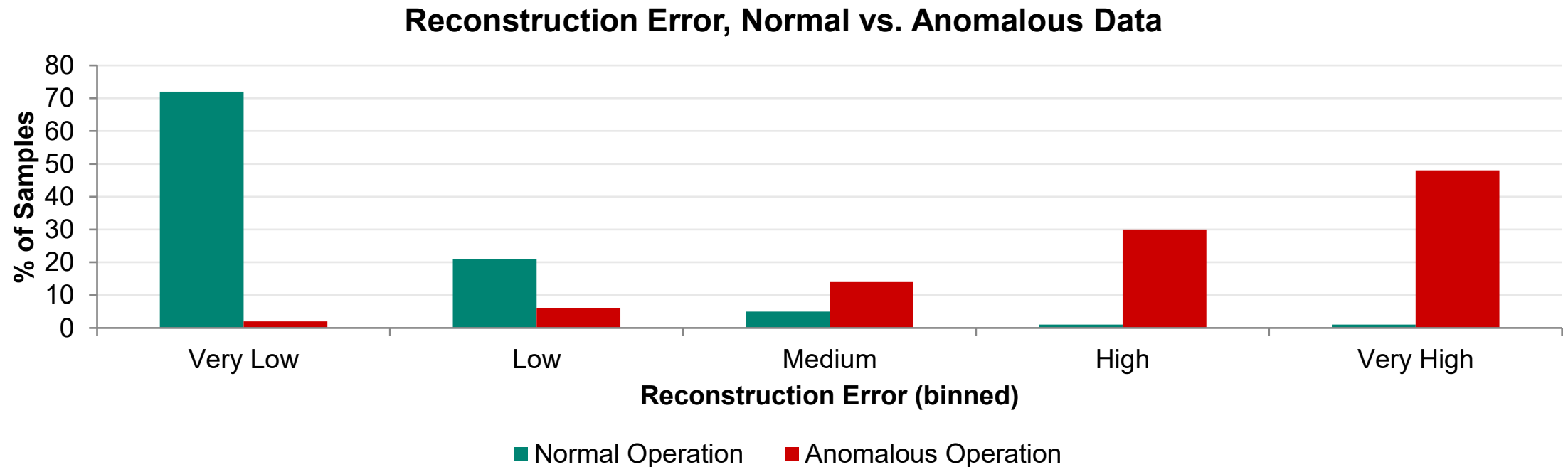
Because the latent space is smooth and continuous, you can sample a brand-new z that was never an encoded input — and decode it into a novel, plausible output. That's generation.

This is conceptually the ancestor of the diffusion models from the LLM session — both learn to map between a simple distribution and complex data.



Anomaly Detection: The Killer App for Engineers

Train only on normal operation — let reconstruction error do the flagging



Normal samples reconstruct well (low error); the model never learned the anomalous pattern, so it reconstructs poorly (high error) — no fault labels required.

Other Engineering Applications

Beyond anomaly detection



Compression

Encode high-dimensional sensor data into a compact latent code for cheaper storage or transmission.



Denoising

Clean up noisy sensor signals by reconstructing the underlying clean version.



Pretraining / Feature Extraction

Learn a useful representation from unlabeled data, then reuse it as input to a smaller supervised model.

Limitations

The honest counterweight



Reconstructions Can Be Blurry

Vanilla autoencoders often average over plausible details rather than committing to sharp ones.



Latent Space Isn't Guaranteed Structured

A plain autoencoder's latent space can have gaps and discontinuities — part of what motivates VAEs.



Risk of Learning the Identity Function

An under-constrained bottleneck can let the network cheat instead of learning real structure.



VAEs Trade Off Sharpness for Structure

A smoother, more usable latent space often comes at the cost of some reconstruction fidelity.